HONEYWELL EMBEDDED ENGINEERING TEAMS — PA · IA · BA

AI Champions Programme

Module 05

Embedded Development — SDD to Complete Software SDLC

Two engineering flavours, one disciplined lifecycle — build a greenfield feature from specification, then evolve a brownfield firmware repository safely.

DURATION

3.5 Hours · Module 5 of 12

FORMAT

Twin-Track Build Lab

DELIVERED FOR

Honeywell Embedded Engineering

NIIT StackRoute × RPS Programme Design

How We'll Spend These Three and a Half Hours

Six connected moves — from problem framing to a side-by-side comparison of both paths.

01

Problem Framing & Architecture

30 min

Frame the problem, design the architecture & interfaces before any code is written.

02

Greenfield: Spec to Implementation

60 min

Build a new embedded feature end to end from Module 04's specification.

03

Testing & Build Validation

30 min

Unit tests, static analysis, build/CI checks on the greenfield feature.

04

Brownfield: Repository Archaeology

45 min

Analyse an existing firmware repository before changing a single line.

05

Safe Evolution & Regression Protection

30 min

Implement a brownfield enhancement with a minimal-change strategy.

06

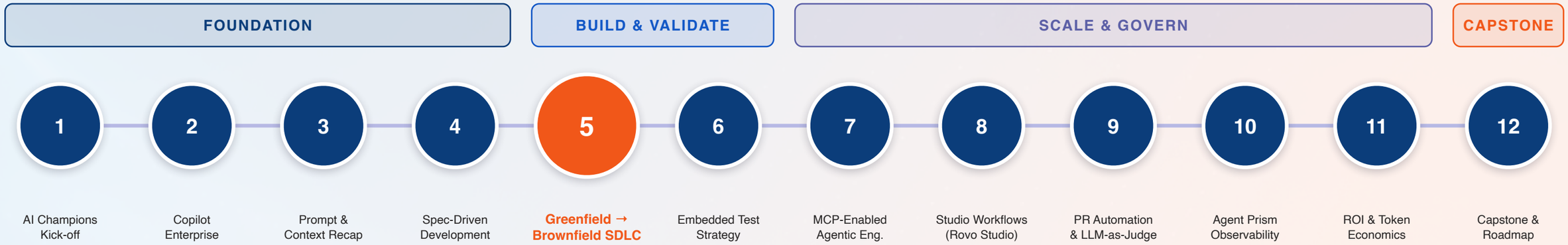
Compare & Release Readiness

15 min

Greenfield vs. brownfield, side by side — and what “done” means for each.

Module 05 in the 12-Module Journey

The longest module in the Foundation-to-Build stretch — where specification becomes working, tested firmware.



Why this matters: Module 06's test strategy validates exactly what gets built here. Module 07's MCP workflows and Module 09's PR quality gates both assume a codebase that went through this disciplined SDLC — greenfield or brownfield.

Two Engineering Flavours, One Disciplined Lifecycle

Neither path is the “easy” one — they optimize for different risks.

GREENFIELD — BUILT FROM SPECIFICATION

- No existing code to preserve — the risk is building the wrong thing well.
- Architecture & interfaces are designed fresh, directly from Module 04's specification.
- Success looks like: complete feature, clean structure, full test coverage from day one.
- Copilot generates most confidently here — clear scope, clean slate, well-represented patterns.
- The discipline that matters: don't let a clean slate become an unscoped one.

BROWNFIELD — SAFE EVOLUTION OF EXISTING CODE

- Existing behaviour must be preserved — the risk is breaking something that already works.
- Architecture & interfaces are discovered through repository archaeology before anything changes.
- Success looks like: minimal, well-scoped change with proven behaviour preservation.
- Copilot needs the most supervision here — undocumented history, tribal knowledge, thin coverage.
- The discipline that matters: understand before you touch, then change as little as possible.

The Complete Software SDLC, Stage by Stage

Seven stages, walked twice today — once for greenfield, once for brownfield.

Problem Framing	Architecture & Interfaces	Module Structure	C/C++ Implementation	Test & Static Analysis	Build & CI/CD	Release Readiness
What are we actually solving, and for whom?	Module boundaries, contracts, HAL touchpoints.	How the feature decomposes into files & functions.	Copilot-assisted, checked against spec at every step.	Unit, integration, and static checks before anything ships.	Toolchain build, pipeline validation, reproducibility.	Everything that must be true before this ships.

What differs between the two paths: Every stage above happens for both flavours — what changes is how much discovery precedes it.

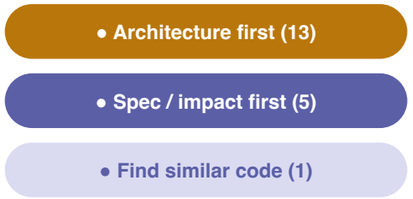
What the Calibration Survey Told Us About Brownfield Work

Real answers from 19 Honeywell engineers — mapped straight onto today's repository-archaeology lab.

Your First Move on an Existing Codebase

13

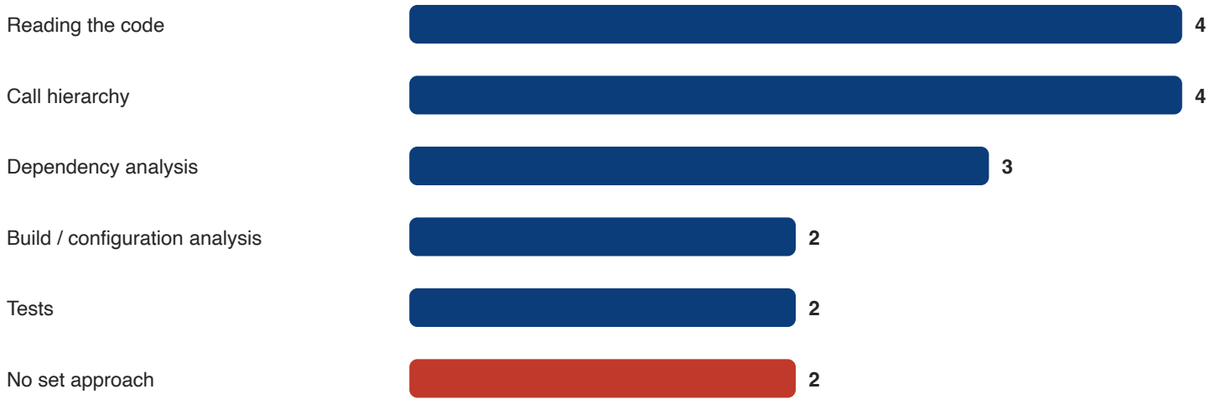
of 19



13 of 19 already start with repository archaeology, unprompted. Today's lab gives that instinct a name — and a structure that scales past one engineer's memory.

What this cohort builds: 8 of 19 work mainly on firmware, 4 on RTOS-based systems — exactly the brownfield territory this module's lab uses.

How Do You Work Out What a Change Will Affect?



The gap this module closes: 2 of 19 say they have no set approach to change-impact analysis — dependency mapping in today's lab gives everyone the same repeatable method.

Repository Archaeology: Before You Touch Brownfield Code

Six investigations, one confident change — the discovery work that makes minimal-change possible.

Architecture Analysis

Map the existing module boundaries and design intent — stated or implied.

Dependency Mapping

Trace what calls what, and what breaks if this function's behaviour changes.

Interface Contracts

Identify every consumer of the interface you're about to touch.

HAL Boundaries

Know exactly where hardware abstraction starts and application logic ends.

Change Impact Analysis

Combine the above into a scoped answer: what does this change actually touch?

Extension Points

Find where the codebase already expects to grow — use those seams first.

This is context engineering again: these six investigations are the History and Design context categories from Module 03's context inventory, applied at repository scale.

The Minimal-Change Strategy

Four principles that keep a brownfield enhancement safe — and reviewable.

Prefer Extension Over Modification

Add new code paths at existing extension points instead of rewriting working logic in place.

Preserve Backward Compatibility

Existing callers, configs & behaviours keep working unless the spec explicitly says otherwise.

Protect Against Regression

Every existing test still passes; new tests cover exactly the new behaviour, nothing more.

Keep Changes Reversible

Small, isolated commits that could be reverted individually if something's wrong.

The tell-tale sign of a good brownfield diff: a reviewer can understand exactly what changed and why, without reading the whole file.

Greenfield Build: From Module 04's Spec to Working Code

The specification you wrote last module becomes this module's architecture, one stage at a time.

Start From the Spec

Requirements, constraints & acceptance criteria from Module 04, unchanged.

Design the Architecture

Module boundaries & interfaces that satisfy every constraint in the spec.

Implement Task by Task

Copilot-assisted, each task checked back against the spec as it lands.

Test Against Acceptance Criteria

Every AC becomes a test — direct, traceable, no guesswork.

No spec to start from? That's a signal to stop and go back to Module 04 — greenfield work without a spec is just prompt-only development with extra steps.

Greenfield vs. Brownfield: The Same Stage, Two Realities

Same seven-stage SDLC from Slide 5 — here's what changes at each stage.

SDLC STAGE	GREENFIELD	BROWNFIELD
Problem Framing	Defined by the spec	Constrained by existing behaviour
Architecture	Designed fresh	Discovered via repository archaeology
Implementation	New modules, new files	Minimal, isolated diffs at extension points
Testing	Full coverage from scratch	New tests + full regression suite
Release Readiness	Feature-complete vs. spec	Behaviour-preserved vs. baseline

Testing & Build Validation Along the Way

A preview, not the full picture — Module 06 goes deep on test strategy next.

Unit & Integration Tests

Run continuously as tasks land, not saved for the end of implementation.

Static Analysis

Catches coding-standard and safety-pattern violations before a human even looks.

Build Validation

Confirms the feature compiles cleanly across the target toolchain, every time.

CI Pipeline Checks

The same checks a reviewer will see — run locally, before the PR ever opens.

Coming next in Module 06: mocks/stubs/fakes, boundary & negative testing, sanitizers, and full specification-to-test traceability.

Hands-On Lab: Build Greenfield, Then Evolve Brownfield

Two connected exercises, same engineer, same afternoon — the contrast is the lesson.

PART 1 GREENFIELD

- Build a greenfield feature from the Module 04 spec, start to finish.
- Design architecture, implement task by task, test against acceptance criteria.

PART 2 BROWNFIELD

- Analyse a brownfield firmware repository — architecture, dependencies, HAL boundaries.
- Create a change plan, implement safely, validate behaviour preservation against the baseline.

Compare Approaches

Debrief: what took longer, what felt riskier, and why.

Instructor Spot-Checks

Facilitators review both diffs against the minimal-change principles.

Carry Forward

Both artefacts feed Module 06's test-strategy lab directly.

Release Readiness: What “Done” Means for Each Path

Two checklists, one shared standard — nothing ships without both columns checked.

GREENFIELD READY TO SHIP

- Every acceptance criterion from the spec has a passing test
- Architecture matches what was designed — no undocumented drift
- Static analysis clean; build passes on the full target toolchain
- Interfaces documented for the next engineer who extends this
- PR describes which spec this implements, ready for Module 09's review

BROWNFIELD READY TO SHIP

- Full regression suite passes — not just tests for the new behaviour
- Diff is minimal and isolated to the scoped extension point
- Every changed interface's callers were checked, not assumed safe
- Behaviour-preservation validated against a documented baseline
- PR explains what was preserved as clearly as what changed

Module 05 Outcomes & What's Next

Everything below should be true before we build the full test strategy in Module 06.

- Can walk the seven-stage SDLC from problem framing to release readiness
- Built a greenfield feature end to end from a Module 04 specification
- Practiced repository archaeology: architecture, dependencies, interfaces, HAL boundaries
- Applied the minimal-change strategy to a real brownfield enhancement
- Can explain how each SDLC stage differs between greenfield and brownfield work
- Validated behaviour preservation against a documented baseline
- Know the release-readiness checklist for both engineering flavours
- Have two real artefacts — greenfield and brownfield — ready for Module 06's test strategy

NEXT — MODULE 06: EMBEDDED TEST STRATEGY, VALIDATION AND REGRESSION ENGINEERING

2.5 hours · Host-based unit testing, mocks/stubs/fakes, boundary & negative testing, static analysis, and full specification-to-test traceability.

Questions & Discussion

Module 05 — Embedded Development: SDD to Complete Software SDLC